KORIAMC

KoriaFactions

Documentation technique · Plugin de factions

Version 1.0.0 · Spigot 1.8.8 · Java 8
Mise à jour le 25 septembre 2026

Sommaire

1. 1. Compilation
2. 2. Architecture
3. 3. Cycle de vie
4. 4. Configuration
5. 5. Modèle de données
6. 6. Stockage
7. 7. Règles de jeu
8. 8. Commandes
9. 9. Menus
10. 10. Tâches planifiées
11. 11. API
12. 12. PlaceholderAPI
13. 13. Étendre le plugin

Plugin de factions PvP pour le serveur **KoriaMC**.

Plateforme	Spigot / PaperSpigot **1.8.8** (NMS `v1_8_R3` par réflexion uniquement)
Java	8
Package racine	`fr.koriamc.factions`
Classe principale	`fr.koriamc.factions.KoriaFactions`
Chargement	`load: POSTWORLD`
Dépendances optionnelles	Vault (+ un plugin d'économie), PlaceholderAPI
Utilisé par	KoriaQuest (API, évènements, points de faction, `/f quests`)

1. Compilation

Projet Maven sans dépendance embarquée (toutes en `provided`) :

```
mvn clean package
```

Le jar est produit dans `target/KoriaFactions-1.0.0.jar`. Seul `plugin.yml` est filtré par Maven (version) ; les JSON sont copiés tels quels.

Sur la machine de développement, Maven n'est pas dans le PATH et travaille hors ligne :

```
JAVA_HOME="C:/Program Files/Java/jdk1.8.0_202" ".../MCP/Serveur/apache-maven-3.9.6/bin/mvn" -o
clean package
```

`spigot-api` est pris avec le classifier `shaded` et des exclusions `*` (le jar contient déjà Gson, Guava et l'API chat).

2. Architecture

Package	Rôle
`(racine)`	`KoriaFactions` : cycle de vie, chargement de la configuration, accès aux services
`api`	`KoriaFactionsAPI` (façade statique pour les autres plugins)
`api.event`	évènements Bukkit publiés par le plugin
`command`	`CommandManager` (commande dynamique), `SubCommand`, `CommandContext`, `MainCommand`
`command.impl`	sous-commandes regroupées par thème (`Core`, `Info`, `Claim`, `Member`, `Relation`, `Teleport`, `Settings`, `Admin`, `Help`)
`config`	lecture / écriture JSON (`JsonFile`, `JsonWriter`), réglages typés (`Settings`, `ProtectionSettings`, `EconomySettings`, `RoleSettings`), `Messages`, `ConfigDocs`
`gui`	menus (`Menu` et ses sous-classes), `MenuManager`, `ChatPrompt`, éditeur `/f config`

Package	Rôle
`hook`	seules classes qui touchent aux API externes : `VaultHook`, `PapiHook`, `FactionsExpansion`
`listener`	écouteurs Bukkit : chat, combat, joueurs, protection, monde
`manager`	logique métier (un service par domaine)
`model`	données persistantes et énumérations
`storage`	persistance JSON / SQLite / MySQL
`util`	texte, objets, matériaux, sons, NMS

Services (`manager`)

Classe	Responsabilité
`FactionManager`	registre des factions, création / dissolution, membres, relations, puissance, raidable, flags, permissions, points, journal
`PlayerManager`	registre des `FPlayer` (par UUID et par nom)
`ClaimManager`	carte des chunks revendiqués, règles de claim / unclaim / overclaim, zones système
`PowerManager`	régénération périodique, régénération hors ligne, perte à la mort
`ProtectionService`	décision « ce joueur peut-il faire cette action ici ? », explosions, feu
`TerritoryManager`	changement de chunk : notifications, auto-claim, carte automatique
`TeleportManager`	home / warps avec délai d'attente, annulation au mouvement ou aux dégâts, délai de réutilisation
`UpgradeManager`	améliorations (définitions de `factions.json`) et leurs effets en jeu
`ChestManager`	coffre partagé (`/f chest`), inventaire sérialisé en Base64 dans la faction
`EconomyManager`	coûts et gains via Vault. **Aucun argent de faction** : tout passe par le compte du joueur
`TopManager`	classements `/f top` (POWER, CLAIMS, MEMBERS, POINTS) mis en cache
`InactivityManager`	expulsion des inactifs, dissolution des factions abandonnées
`MapManager`	rendu de `/f map`
`SeeChunkManager`	particules de bordure de chunk
`PlaceholderService`	variables `{cle}` internes (joueur et faction)
`CooldownManager`, `ConfirmationManager`	délais par joueur, confirmation des actions dangereuses

Chaque service est créé une seule fois dans `onEnable` et accessible par un accesseur de `KoriaFactions` (`plugin.factions()`, `plugin.claims()` ...). `KoriaFactions.get()` renvoie l'instance (null si désactivé).

3. Cycle de vie

Démarrage (`onEnable`)

1. Création des `JsonFile` (avec leurs sections libres) et de tous les services.
2. Avertissement si d'anciens fichiers `.yml` sont présents (ils ne sont plus lus).
3. `loadConfigs()` : lecture des 5 fichiers, fusion de `config.json` + `factions.json` + `territoire.json` en une seule `Configuration`, chargement des réglages typés, des messages et des améliorations, déclaration des permissions de grade.
4. `ConfigDocs` : lecture de `descriptions.json` (jar) et écriture de `LISEZMOI.txt`.
5. `DataManager.init()` puis `load()`. **En cas d'échec, le plugin se désactive** pour ne pas écraser les données.
6. Vault (retenté un tick plus tard pour les économies chargées après), PlaceholderAPI.
7. Enregistrement des écouteurs, puis de la commande principale dans le `CommandMap`.
8. Tâches planifiées (sauvegarde auto, puissance, inactivité, seechunk), mise à jour du territoire des joueurs en ligne.

Arrêt (`onDisable`)

Désenregistrement de la commande, arrêt des tâches, annulation des téléportations, fermeture des menus et des coffres, désenregistrement PAPI, puis **sauvegarde synchrone** et fermeture du stockage.

Rechargement (`/f reload` ou éditeur)

`KoriaFactions.reload()` relit la configuration, réapplique les réglages des zones système, recharge les commandes, le chat et Vault, redémarre les tâches, vide le cache des classements, rafraîchit les coffres et recalcule la puissance et l'état raidable. Le type de stockage et le nom de la commande principale demandent un redémarrage.

4. Configuration

Fichiers

Fichier	Racines
`config.json`	`mondes-desactives` , `debug` , `stockage` , `commande` , `commandes` , `economie` , `chat` , `classement` , `journal` , `annonces` , `sons`
`factions.json`	`factions` , `roles` , `permissions` , `flags` , `relations` , `puissance` , `pvp` , `home` , `warps` , `teleportation` , `coffre` , `ameliorations` , `inactivite`
`territoire.json`	`claims` , `zones` , `protection` , `explosions` , `blocs` , `entites` , `notifications` , `carte` , `seechunk`
`messages.json`	tous les textes (clés en anglais)
`menus.json`	`principal` , `sans-faction` , `membres` , `permissions` , `flags` , `ameliorations` , `warps` , `editeur`

Les trois fichiers de réglages sont fusionnés dans une même vue : **leurs clés racines doivent rester uniques**.

`JsonFile`

- Au premier lancement, le fichier du jar est copié dans le dossier du plugin.
- Chaque option absente du fichier du serveur prend la valeur du jar (défauts de la `MemoryConfiguration`). Les options manquantes sont listées dans la console au chargement.
- **Sections libres** (ex. `ameliorations.liste`, `chat.grades`, `principal.objets`) : leurs entrées ne sont pas complétées par le jar, ce qui permet d'en supprimer ou d'en ajouter. Elles sont déclarées dans `onEnable`.
- Erreur de syntaxe : message en français avec ligne et colonne. Au premier chargement, les valeurs du jar sont utilisées ; lors d'un rechargement, l'ancienne version reste active.
- Écriture (éditeur) : fichier temporaire puis remplacement, format compact de `JsonWriter` (130 colonnes).
- Nombres : entier si pas de partie décimale, sinon `double`.

Réglages typés

`Settings`, `ProtectionSettings`, `EconomySettings` et `RoleSettings` lisent la `Configuration` fusionnée et exposent des champs publics (`plugin.settings().powerPerClaim` ...). Tout nouveau réglage doit y être ajouté, avec :

1. la valeur par défaut dans le JSON du jar (clé française avec tirets) ;
2. une entrée dans `descriptions.json` (texte, et valeurs possibles pour un choix) ;
3. sa lecture dans la classe de réglages concernée.

Matériaux

`MaterialSet` et `Items.parse` acceptent le nom Bukkit (`CHEST`), l'ID (`54`) ou `ID:data` (`35:14`) : c'est ce qui permet de cibler les blocs moddés de KoriaMC.

Messages

`Messages` lit `messages.json`. Les variables sont passées par paires (`send(joueur, "cle", "player", nom)`), `{prefix}` est remplacé partout. Une valeur peut être une chaîne ou une liste de lignes ; une ligne commençant par `[actionbar]` va dans la barre d'action, `[title]titre;;sous-titre` affiche un titre (paquets NMS par réflexion, repli sur l'API Bukkit) ; `""` n'envoie rien.

Éditeur en jeu (`/f config`)

`ConfigFilesMenu` liste les fichiers, `ConfigSectionMenu` affiche une section. Le type d'une option détermine son modèle (`menus.json` → `editeur.types`) : `section`, `section-libre`, `oui`, `non`, `nombre`, `texte`, `choix` (si `descriptions.json` donne des valeurs), `liste`, `complexe` (non modifiable). Actions : `OUVRIR`, `BASCULER`, `AUGMENTER:n`, `DIMINUER:n`, `SAISIR` (saisie au chat via `ChatPrompt`), `CHOIX-SUIVANT/PRECEDENT`, `AJOUTER`, `RETIRER`, `REMPLACER`, `DEFAUT`, `AJOUTER-ENTREE` (copie la dernière entrée d'une section libre). Chaque modification écrit le fichier puis appelle `reload()`. Les options dont le nom contient `mot-de-passe` sont masquées.

5. Modèle de données

`Faction`

Champ	Type	Contenu
`id`	String	identifiant stable (les zones système ont `wilderness`, `safezone`, `warzone`)
`tag`	String	nom affiché, modifiable (`/f rename`)
`description`, `motd`	String	
`createdAt`, `lastRenamed`	long	horodatages (ms)
`home`	`SerialLoc`	
`warps`	Map<String, `Warp`>	nom, position, mot de passe, auteur, date
`relationWishes`	Map<id, Relation>	souhaits d'alliance (seul `ALLY` est conservé)
`relationChanges`	Map<id, Long>	date du dernier changement (délai entre changements)
`flags`	Map<String, Boolean>	valeurs propres à la faction (sinon défaut de `factions.json`)
`permissions`	Map<sujet, Map<action, Boolean>>	permissions par rôle / alliés
`playerPermissions`	Map<UUID, Map<action, Boolean>>	permissions personnelles d'un membre (prioritaires sur son rôle)
`invites`	Map<UUID, Long>	invitations et leur expiration
`bans`	Set	
`upgrades`	Map<id, Integer>	niveau de chaque amélioration
`points`	long	points de faction (quêtes...), jamais négatifs
`chest`	String	contenu du coffre en Base64
`logs`	List<`LogEntry`>	journal (type + variables, rendu à l'affichage)

`members` et `raidable` sont `transient` : l'appartenance est portée par `FPlayer.factionId` et l'index des membres est reconstruit au chargement. `ensureInit()` corrige les collections nulles et purge les valeurs obsolètes (relations autres qu'ALLY, permissions inconnues).

`FPlayer`

Persistant : `uuid`, `name`, `factionId`, `role`, `title`, `power`, `cachedMaxPower`, `firstJoin`, `lastSeen`, `lastFactionLeave`, `joinedFactionAt`, `kills`, `deaths`, `chatMode`, `territoryNotifications`, `autoMap`.
Temporaire : `bypass`, `spy`, mode d'auto-claim, dernier chunk / territoire, seechunk, anti-spam des refus.

`Claim`

Clé : `ChunkPos.key()` = `monde;x;z`. Valeur : `factionId`, `claimedAt`, `claimedBy`, `accessPlayers`, `accessFactions` (accès accordés par `/f access`).

Énumérations

Enum	Valeurs
`Role`	`RECRUIT`, `MEMBER`, `OFFICER`, `COLEADER`, `LEADER`
`Relation`	`ENEMY`, `ALLY`, `MEMBER` (pas de neutre ni de trêve : ennemis par défaut)
`TerritoryType`	`WILDERNESS`, `SAFEZONE`, `WARZONE`, `OWN`, `ALLY`, `ENEMY`
`FactionFlag`	`OPEN`, `PEACEFUL`, `PERMANENT`, `EXPLOSIONS`, `FRIENDLY_FIRE`, `PVP`, `MONSTERS`, `FIRE_SPREAD`, `INFINITE_POWER`
`PermAction` (territoire)	`BUILD`, `DESTROY`, `CONTAINER`, `DOOR`, `BUTTON`, `LEVER`, `PRESSURE_PLATE`, `USE_ITEM`, `INTERACT`, `ENTITY_INTERACT`, `SPAWNER`
`PermAction` (gestion)	`HOME`, `WARP`, `INVITE`, `KICK`, `BAN`, `CLAIM`, `UNCLAIM`, `UNCLAIM_ALL`, `PROMOTE`, `DEMOTE`, `RELATION`, `SETHOME`, `SETWARP`, `DELWARP`, `CHEST`, `DESCRIPTION`, `MOTD`, `TITLE`, `FLAGS`, `UPGRADE`, `ACCESS`, `ANNOUNCE`, `LOGS`, `RENAME`, `PERMS`
`PermSubject`	un rôle (hors chef) ou `ALLY` ; les ennemis n'ont aucune permission
`UpgradeType`	`MEMBERS`, `WARPS`, `CHEST`, `POWER`, `CROPS`, `EXP`, `SPAWNERS`, `DAMAGE_REDUCTION`
`ExplosionType`	`TNT`, `CREEPER`, `FIREBALL`, `WITHER`, `OTHER`
`ChatMode`	`PUBLIC`, `FACTION`, `ALLY`, `MOD`

6. Stockage

Choisi par `stockage.type` (`JSON`, `SQLITE`, `MYSQL`), redémarrage nécessaire pour en changer.

Principe (`DataManager`)

- Tout est en mémoire. Chaque entité est sérialisée en JSON par Gson (`disableHtmlEscaping`).
- **Sauvegarde** : `snapshot()` sérialise tout sur le thread principal (et enregistre les coffres ouverts), puis un thread unique `KoriaFactions-Save` écrit le snapshot. Sauvegarde automatique toutes les `stockage.sauvegarde-auto-minutes` minutes, et sauvegarde synchrone à l'arrêt (attente max. 30 s des écritures en cours).
- Tant que le chargement n'a pas réussi, rien n'est écrit.
- Une entrée illisible est ignorée avec un avertissement, sans bloquer le chargement.

JSON

`plugins/KoriaFactions/data/factions.json`, `players.json`, `claims.json` : un objet `{ id: données }` par fichier. Écriture atomique (fichier `.tmp` puis déplacement), fichier non réécrit s'il n'a pas changé, copies tournantes dans `data/backups/` (`stockage.copies-de-securite`). `stockage.json-indente` pour des fichiers lisibles.

SQL (SQLite / MySQL)

Trois tables `<prefixe>factions`, `<prefixe>players`, `<prefixe>claims`, chacune `(id PRIMARY KEY, data TEXT/LONGTEXT)` contenant le même JSON. Seules les lignes modifiées depuis la dernière sauvegarde sont écrites (upsert en lot), les lignes disparues sont supprimées, le tout dans une transaction. Pilotes fournis par Spigot 1.8.8

(`org.sqlite.JDBC` , `com.mysql.jdbc.Driver` , compatible MariaDB). Reconnexion automatique si la connexion MySQL tombe.

Ajouter un champ à `Faction` ou `FPlayer` suffit à le persister (Gson). Initialiser les collections dans `ensureInit()` pour les anciennes données.

7. Règles de jeu

Relations

Toutes les factions sont ennemies par défaut. Une alliance existe quand **chacune** a `ALLY` dans ses `relationWishes` envers l'autre. `/f ally` propose ou accepte, `/f enemy` rompt ou refuse. Nombre d'alliés limité et délai entre changements (`relations`). Un joueur sans faction est l'ennemi de tous.

Permissions (`FactionManager.hasPermission`)

1. Joueur en bypass : autorisé.
2. Territoire non normal : refusé (géré ailleurs par `ProtectionService`).
3. Membre de la faction : le **chef a tout** ; sinon la permission personnelle du joueur (`/f perms <joueur>`) si elle existe et n'est pas verrouillée ; sinon celle de son rôle.
4. Allié : permission du sujet `ALLY` . Ennemi : rien.

Les valeurs par défaut et les permissions verrouillées (non modifiables par les factions) viennent de `factions.json` .

Protection (`ProtectionService.canAct`)

Ordre de décision pour une action à une position :

1. Monde désactivé ou joueur en bypass : autorisé.
2. **Wilderness** : règles `zones` (construire, détruire, utiliser, liste blanche d'utilisation, mondes interdits).
3. **SafeZone / WarZone** : permission `koriafactions.zone.<id>.build` , sinon règles de la zone.
4. **Territoire de faction** : accès du chunk (`/f access`) pour les actions autorisées, puis permissions de faction, puis actions autorisées sur une faction **raidable** (`protection` → actions permises aux ennemis quand raidable).
5. Refus : message (anti-spam), et dégâts optionnels (« pain build »).

Les écouteurs (`ProtectionListener` , `WorldListener`) traduisent chaque évènement Bukkit en `PermAction` . Explosions et propagation du feu ont leurs propres règles par type de zone (`explosions` , flags `EXPLOSIONS` et `FIRE_SPREAD`).

Puissance et raidable

- Chaque joueur a une puissance entre un minimum et un maximum (maximum configurable par grade via permission).
- `PowerManager` régénère toutes les `regen` secondes (en ligne), applique la régénération hors ligne à la connexion, et retire de la puissance à la mort (`PowerLossEvent` , modifiable).
- Puissance de faction = somme des membres (en ligne ou non selon la config) + amélioration `POWER` , plafonnée.

- Chunks gardés = puissance / `claims.puissance-par-claim`. **Raidable** quand la faction a plus de claims que sa puissance ne le permet (sauf flag `INFINITE_POWER`). `checkRaidable` prévient la faction à chaque changement d'état.

Territoire

`ClaimManager.claim` vérifie : monde autorisé, permission `CLAIM`, limite de puissance et plafond, connexion aux claims existants, zone tampon, distances au spawn / SafeZone / WarZone, ennemi proche (les claims sont gratuits). L'**overclaim** prend un chunk à une faction raidable (options : bordure seulement, sans connexion, prévenir la victime, annonce). La perte d'un chunk peut supprimer le home et les warps qui s'y trouvaient.

Améliorations

Définies dans `factions.json` (`ameliorations.liste`, section libre : **ce qui est dans le fichier s'applique, sans compléter avec le jar**). Chaque amélioration a un type (`UpgradeType`), une case de menu et des niveaux (prix Vault, valeur). Effets :

Type	Effet de la valeur
`MEMBERS`, `WARPS`	membres / warps maximum en plus
`CHEST`	lignes du coffre débloquées (0 à 6)
`POWER`	puissance ajoutée à la faction
`CROPS`	chance (%) qu'une culture pousse d'une étape supplémentaire dans le territoire
`EXP`	expérience (%) en plus sur les mobs tués dans le territoire
`SPAWNERS`	chance (%) qu'un spawner du territoire fasse apparaître un mob supplémentaire. Ce mob apparaît avec la raison `CUSTOM` et porte la métadonnée `UpgradeManager.SPAWNER_METADATA` (`koriafactions-spawner`), que KoriaQuest traite comme un mob de spawner
`DAMAGE_REDUCTION`	réduction (%) des dégâts subis par un membre dans son territoire

Les erreurs de définition sont remontées au rechargement.

Économie

Aucun solde de faction. Coûts (création, claim, warp...) et achats d'améliorations sont prélevés sur le compte Vault du joueur ; `koriafactions.bypass.cost` dispense de payer. Économie activée sans Vault : les actions payantes sont **refusées** (jamais gratuites).

Autres

- **Chat** : tag de faction et couleur du grade serveur (liste de permissions dans `chat.grades`) dans le chat public, canaux `FACTION`, `ALLY`, `MOD`, espion admin. Priorité d'écoute configurable.
- **Téléportations** : délai d'attente annulé au mouvement / aux dégâts, délai de réutilisation, home et warps instantanés dans son propre claim sans ennemi proche. Contournements : `koriafactions.bypass.warmup`, `koriafactions.bypass.cooldown`.
- **Inactivité** : vérification périodique, expulsion des membres inactifs, dissolution des factions abandonnées (sauf flag `PERMANENT`).

- **Journal** : `FactionManager.log(f, type, variables)` ; le texte est rendu à l'affichage depuis `logs.entries.<TYPE>`.

8. Commandes

Fonctionnement

- La commande principale (`/f` par défaut) est une `MainCommand` ajoutée au `CommandMap` par réflexion : son nom et ses alias viennent de `config.json` (`commande`), elle n'est pas déclarée dans `plugin.yml`.
- Chaque sous-commande hérite de `SubCommand` (id, joueur uniquement, nombre d'arguments minimum) et lit dans `config.json` → `commandes.<id>` : `actif`, `alias`, `delai` (secondes), `permission`. L'usage et la description de l'aide sont dans `messages.json` → `help.commands.<id>`.
- Permissions déclarées dynamiquement : `TRUE` par défaut pour les commandes joueur, `OP` pour celles marquées `admin()`. `withAdminPermission()` ajoute `koriafactions.admin.<id>` pour agir sur une autre faction.
- `dispatch` vérifie dans l'ordre : sous-commande connue, permission, joueur requis, arguments, monde désactivé, délai (`koriafactions.bypass.cooldown`), puis exécute ; une exception est journalisée et le joueur prévenu.
- Sans argument : aide, ou menu principal si `commande.sans-argument` vaut `MENU`.
- `quests` : raccourci vers un autre plugin. Le joueur exécute la commande de `commandes.quests.commande` (`quetes faction` par défaut, les quêtes de faction de KoriaQuest) si elle existe sur le serveur (`CommandManager.isKnownCommand`), sinon il reçoit `quests.unavailable`. KoriaFactions ne dépend donc pas de KoriaQuest. Le menu principal a un objet `quetes` qui appelle `/f quests`.

Liste

Sous-commande	Alias par défaut	Permission
help	aide, ?	koriafactions.command.help
menu	gui, m	koriafactions.command.menu
create / disband / rename	creer / dissoudre / tag, renommer	koriafactions.command.create / disband / rename
desc / motd	description	koriafactions.command.desc / motd
show / list	who, info, i / liste, ls	koriafactions.command.show / list
map	carte	koriafactions.command.map
claim / claimat	revendiquer, cl	koriafactions.command.claim
unclaim / unclaimall	uc / uca	koriafactions.command.unclaim / unclaimall
autoclaim / autounclaim	aclaim / aunclaim	koriafactions.command.autoclaim / autounclaim
invite / deinvite / invites	inv / uninvite / invitations	koriafactions.command.invite / deinvite / invites
join / leave / kick	rejoindre / quitter / expulser	koriafactions.command.join / leave / kick
ban / unban / banlist	bannir / debannir / bans	koriafactions.command.ban / unban / banlist

Sous-commande	Alias par défaut	Permission
promote / demote / leader / title	up / down / chef / titre	koriafactions.command.promote / demote / leader / title
ally / enemy / relations	allie / ennemi / rel	koriafactions.command.relation / relations
home / sethome / delhome		koriafactions.command.home / sethome / delhome
warp / warps / setwarp / delwarp	w / / sw / dw	koriafactions.command.warp / setwarp / delwarp
chat	c, ch, canal	koriafactions.command.chat
chest	coffre, pv	koriafactions.command.chest
seechunk	sc, border	koriafactions.command.seechunk
top / power	classement / pow, p	koriafactions.command.top / power
quests	quetes, quete, q	koriafactions.command.quests
perms / flags / open	permissions / flag / ouvrir	koriafactions.command.perms / flags / open
access / logs / announce	acces / journal / annonce	koriafactions.command.access / logs / announce
coords / toggle / version	pos / basculer / ver	koriafactions.command.coords / toggle / version
bypass / spy	admin / chatspy	koriafactions.admin.bypass / spy
safezone / warzone	sz / wz	koriafactions.admin.safezone / warzone
setpower / addpower		koriafactions.admin.power
setpoints / addpoints		koriafactions.admin.points
forcejoin / forcekick / setleader	fjoin / fkick	koriafactions.admin.forcejoin / forcekick / setleader
setflag / setupgrade		koriafactions.admin.setflag / setupgrade
config / save / reload	cfg / sauvegarder / rl	koriafactions.admin.config / save / reload

Permissions spéciales

Permission	Effet
`koriafactions.bypass.cooldown`	ignore les délais de commande et de téléportation
`koriafactions.bypass.warmup`	téléportation sans délai d'attente
`koriafactions.bypass.cost`	actions payantes gratuites
`koriafactions.bypass.commands`	ignore les commandes bloquées par territoire (`protection.commandes-bloquees`)
`koriafactions.chat.color`	codes couleur `&` dans les messages de chat
`koriafactions.zone.<safezone|warzone>.build`	construire dans une zone système
`koriafactions.admin.<id>`	agir sur une autre faction avec la sous-commande `<id>`
permissions de grade (`chat.grades` , `puissance.max-par-grade`)	déclarées `FALSE` par défaut, même pour les OP

9. Menus

`Menu` est un `InventoryHolder` : titre, lignes, fond, objets et effet de **chaque type de clic** viennent de `menus.json`, rien n'est imposé par le code. Un objet se décrit par `case`, `materiau`, `quantite`, `nom`, `description`, `brillant`, `tete` et des clés de clic : `action`, `clic`, `clic-gauche`, `clic-droit`, `clic-milieu`, `shift-clic`, `shift-clic-gauche`, `shift-clic-droit`, `touche-q`, `ctrl-q` (la plus précise l'emporte).

Actions communes : `COMMANDE:`, `CONSOLE:`, `MENU:<id>`, `MESSAGE:`, `FERMER`, `ACTUALISER`, `PAGE-SUIVANTE`, `PAGE-PRECEDENTE`. Actions propres : `BASCULER` (flags, permissions), `ACHETER` (améliorations), `REINITIALISER` (permissions d'un joueur), verbes de l'éditeur.

Identifiants de `MENU:` : `principal`, `membres`, `permissions`, `permissions:<ROLE|ALLY>`, `permissions-joueurs`, `permissions-joueur:<pseudo>`, `flags`, `ameliorations`, `warps`, `config`, `config:<fichier>`.

Les clics sont traités au tick suivant (ouvrir un inventaire pendant un clic désynchronise le client 1.8) ; tous les clics et glissements dans un menu sont annulés. Dans une description, une variable dont la valeur est une liste produit une ligne par élément, et une ligne ne contenant qu'une variable vide disparaît.

10. Tâches planifiées

Tâche	Période
Sauvegarde automatique	`stockage.sauvegarde-auto-minutes`
Régénération de puissance	intervalle de `puissance` (secondes)
Inactivité	intervalle de `inactivite` (minutes), première vérification après 1 minute
Seechunk	particules périodiques pour les joueurs concernés
Téléportations en attente	chaque seconde

Toutes sont arrêtées et relancées par `reload()`.

11. API

`KoriaFactionsAPI` (statique)

Méthode	Rôle
`isAvailable()`	le plugin est-il activé
`getPlayer(Player)`	`FPlayer` (créé si besoin)
`getFaction(Player)`	faction du joueur ou null
`getFactionByTag(String)`, `getFactionById(String)`, `getFactions()`	recherche / liste des factions normales
`getFactionAt(Location)`	propriétaire du chunk (jamais null : wilderness)
`getTerritoryType(Player, Location)`	`TerritoryType` vu par le joueur

Méthode	Rôle
`getRelation(Player, Player)`	`Relation`
`isInOwnTerritory`, `isInSafeZone`, `isInWarZone`	tests de position
`getPower(Player)`, `getFactionPower(Faction)`, `isRaidable(Faction)`	puissance
`getPoints(Faction)`, `addPoints(Faction, long, raison)`, `setPoints(...)`, `getPointsRank(Faction)`	points de faction

`addPoints` renvoie le gain réellement appliqué (0 si l'évènement est annulé ou la faction n'est pas une faction normale) ; le total ne descend jamais sous 0.

Évènements (`api.event`)

Tous héritent de `FactionEvent` (annulables, `getFaction()`).

Évènement	Moment	Particularités
`FactionCreateEvent`	avant création	
`FactionDisbandEvent`	avant dissolution	`Reason` : `COMMAND`, `ADMIN`, `EMPTY`, `INACTIVITY` (`SEASON` n'est plus utilisé) ; annulation ignorée si la dissolution est forcée
`FactionMembershipEvent`	avant entrée / sortie	`Action` : `JOIN`, `LEAVE`, `KICK`, `FORCED_JOIN`, `FORCED_LEAVE` ; annulation ignorée pour les forcées
`FactionRelationEvent`	avant changement de relation souhaitée	
`LandClaimEvent`	avant claim (overclaim compris)	`getChunk()`, `getPlayer()` (peut être null), `isOverclaim()`
`LandUnclaimEvent`	avant unclaim par un joueur	
`PowerLossEvent`	avant perte de puissance à la mort	`setLoss(double)`
`FactionPointsChangeEvent`	avant changement de points	`setAmount(long)`, `getReason()` (`"admin"`, `"koriaquest"`…)

12. PlaceholderAPI

Identifiant `koriafactions`, expansion persistante.

- `%koriafactions_<variable>%` : `player`, `faction`, `role`, `role_prefix`, `role_color`, `title`, `power`, `max_power`, `faction_power`, `faction_max_power`, `faction_points`, `claims`, `max_claims`, `members`, `members_online`, `max_members`, `kills`, `deaths`, `kdr`, `territory`, `territory_color`, `balance`, `world`, `x`, `y`, `z`, `online_players`, `raidable`, `chat_mode`, `label`.
- `%koriafactions_top_<categorie>_<rang>_name%` et `_value` : classements (`POWER`, `CLAIMS`, `MEMBERS`, `POINTS`).
- `%rel_koriafactions_relation%`, `%rel_koriafactions_relation_color%` : relation entre deux joueurs.

Les mêmes variables existent en interne sous la forme `{cle}` dans les messages et menus (`PlaceholderService`).

13. Étendre le plugin

Besoin	Où intervenir
Nouvelle option	JSON du jar + `descriptions.json` + classe de réglages ; section libre à déclarer dans `onEnable` si les entrées sont ajoutables
Nouveau texte	`messages.json` (clé anglaise), envoi via `plugin.msg().send(...)`
Nouvelle sous-commande	classe dans `command.impl` + `register(...)` dans `CommandManager` + entrée `commandes.<id>` (config) + `help.commands.<id>` (messages)
Nouveau menu	sous-classe de `Menu` (`section()` , `render()`), section dans `menus.json` , identifiant dans `MenuManager.open`
Nouvelle action de permission	valeur de `PermAction` (booléen « territoire ») + défauts dans `factions.json` + nom affiché dans les messages
Nouveau type d'amélioration	`UpgradeType` + effet dans `UpgradeManager`
Nouvelle catégorie de classement	`TopManager.CATEGORIES` + `value()` + `top.categories.<CAT>` dans les messages
Donnée persistante	champ dans `Faction` / `FPlayer` (Gson), initialisation dans `ensureInit()`

Conventions : clés d'options en français avec tirets, clés de messages en anglais, commentaires en français, aucune fonctionnalité de vol (serveur PvP). Fonctionnalités retirées volontairement : banque et argent de faction, trêve et neutralité, scoreboard, saisons, powerboost, stats de faction, valeur de faction, `/f stuck` .